Lab 2 Instructions

PUBH 523/623

Nicky Wakim

Directions

Please turn in your `.qmd` and `.html` file on Brightspace. Please use the same naming convention as practice assignments: `lastname_firstinit_lab_02.qmd` and `lastname_firstinit_lab_02.html`.

You can download the `.qmd` file for this lab here.

The above link will take you to your **editing** file. **Please do not remove anything from this editing file!!** You will only add your code and work to this file.

Purpose

The purpose of this lab is to prepare our data for further investigation using several data transformations. We will filter to our target subpopulation, select only the variables relevant to our project, give those variables readable names, and investigate and handle missing data. By the end, you'll have a clean dataset that is ready for building your composite score(s) in Lab 3. **This is meant to take your practice on data transformation one step further: translating written human language to the needed action, function, and arguments.**

This lab uses Lessons 16-25 (piping, import/export, key operators, data transformations, and both missing data lessons). Everything you need has been covered in those lessons.

Before you start

How to use these instructions:

- Each section ends with a red **Task** box. Each task box is copied into your editing file, and your code and writing go right below it.

- Inside each section, the **numbered steps** walk you through the task in order. Do them one at a time and run your code after each step. It's much easier to find a mistake after one step than after five!
- **Use the exact object names given in each task** (like `wbns_01_pop`). Labs 3 and 4 pick up from these names, and it makes it much easier for me to help you when you get stuck.
- When the instructions are different for each project, you will see **tabs** like the ones below. Click on your project's tab. Once you pick a tab, every set of tabs on this page will switch to your project.

Project 1

You will see Project 1 (food support and food security) instructions here.

Project 2

You will see Project 2 (trust and fair treatment in healthcare) instructions here.

Project 3

You will see Project 3 (benefits and mental health) instructions here.

Functions you'll use in this lab

Each function name links to the lesson slide where it's taught, in case you want a refresher.

Function	What it does	Lesson
`import()`	Read a data file into R	Lesson 17
`here()`	Build a file path that starts at your R project folder	Lesson 12
`dim()`	Print the number of rows and columns	Lesson 16
`tabyl()`	Count how many rows fall in each category	Lesson 25
`filter()`	Keep rows that meet a condition	Lesson 21
`filter_out()`	Remove rows that meet a condition, keeping NAs	Lesson 21
`select()`	Keep only certain columns	Lesson 22
`starts_with()`	Select columns whose names start with some text	Lesson 23
`rename()`	Give columns new names	Lesson 22

`glimpse()`	Look at every column's name, type, and first values	Lesson 17
`n_miss()`	Count missing values	Lesson 24
`prop_miss()`	Proportion of values that are missing	Lesson 24
`miss_var_summary()`	Count and percent missing for each variable	Lesson 24
`gt()`	Print a table nicely	Lesson 24
`vis_miss()`	Plot where the NAs are across the dataset	Lesson 24
`theme()`	Adjust how a plot looks (e.g., angle the labels)	Lesson 24
`gg_miss_upset()`	Plot which variables are missing together	Lesson 24
`gg_miss_fct()`	Plot missingness across groups of another variable	Lesson 24
`replace_with_na()`	Turn specific values (like "Refused") into NA	Lesson 25
`drop_na()`	Remove rows with NA in the columns you list	Lesson 25
`export()`	Save data to a file	Lesson 17

Keep the WBNS questionnaire and codebook open while you work. You will look up response options, and which respondents were asked each question, many times in this lab.

Lab activities

The following table will help you identify the appropriate population and variables based on your selected project question from Lab 1. You will build the composite score(s) in the last row in Lab 3, but it helps to know now which variables they need.

	Project 1	Project 2	Project 3
Project Question	For individuals under the federal poverty line, how do food support systems help individuals' food security?	For individuals who identify as LGBTQ, how does trust and/or fair treatment in healthcare affect self-reported health?	For individuals with school-aged children and below the federal poverty line, how do benefits received affect their mental health?

	Project 1	Project 2	Project 3
Population	People under 100% federal poverty line (FPL) and received free groceries or meals in the past 12 months	People who identify as LGBTQ and have a healthcare provider. (You can change this population if you want. For example, you may want to focus on how trust and fair treatment may improve health for people with larger bodies. Check out Q70F for options!)	People who have at least one child (aged 5-18 years old) in the household and are under the 100% FPL
Population variables	`FPL_LT50`, `FPL50_100`, `Q25`, `Q25A`, `Q26`, `Q26A`	`LGBTQ`, `Q108`	`ANY_CHILDHH_5_18`, `FPL_LT50`, `FPL50_100`
Main variables	`FOOD_INSEC`, `Q26D_1`, `Q26D_2`, `Q26D_3`, `Q26D_4`	`Q108B_2`, `Q108B_1`, `Q27`	`Q30_1`, `Q30_2`, `Q30_3`, `Q30_4`, `Q30_5`, `Q30_6`, `Q53_A`, `Q53_B`, `Q53_C`, `Q53_D`, `Q53_E`, `Q53_F`, `Q53_G`, `Q53_H`, `Q53_I`, `Q11`, `Q31_4`
Additional variables	`Q23`, `PPETHM`, `Q37`, `NUM_CHILD_HH`	`Q70F_4`, `Q70F_5`, `PPETHM`, `Q28O`	`NUM_CHILD_HH`, `Q1`, `Q37`, `PPETHM`, `Q27`
Composite score	Composite score of ease of access to free groceries or meals (combined score from `Q26D_`)	Level of trust, courtesy, and respect around treatment (combined score from `Q108B_1` and `Q108B_2`)	Total mental health (combined scores from `Q30_`), Total benefits (combined scores from `Q53_`, `Q11`, and `Q31_4`)

Here is a map of the objects you will create in this lab. Each one is built from the one before it:

Section	Object you create	What it holds
Import your saved data	`wbns_00`	The full dataset you saved in Lab 1
Filter to your population	`wbns_01_pop`	Only the respondents in your population
Select project variables	`wbns_02_pop_vars`	Only the variables for your project
Rename variables	`wbns_03_renamed`	Same data, with readable variable names

Section	Object you create	What it holds
Explore missing data	*(no new dataset)*	Missingness tables and plots
Handle missing data	`wbns_04_na_recoded`, then `wbns_04_no_na`	Refusals turned into `NA`, then rows with missing key variables removed

Import your saved data from Lab 1

At the end of Lab 1, you saved your dataset as an `.rds` file in `data_project/data_processed`. Now we'll bring it back into R.

1. Find your file in the `data_project/data_processed` folder and note its exact name (including `.rds`).
2. Use `import()` from the `rio` package with `here()` to build the file path. Your code will look something like this (replace the file name with yours):

```
wbns_00 <- import(here("data_project", "data_processed", "wbns_data.rds"))
```

3. Use `dim()` to print the number of rows and columns. The first number is rows (respondents) and the second is columns (variables).

! Task

From the exported data file you created in Lab 1, import your data into R using `import()` from the `rio` package. Save this as a new object called `wbns_00`. Use `dim()` to print the number of rows and columns to verify the import.

Tip

If you get an error like `No such file`, double-check: (1) you opened your `.Rproj` before running code, (2) the folder names in `here()` match your folders exactly, and (3) the file name, including `.rds`, is spelled exactly right.

Filtering to Your Target Subpopulation

To implement an asset-focused analysis, we must first isolate the subpopulation relevant to your project using `filter()`.

1. Look up each population variable in the questionnaire. Then use `tabyl()` on each one in `wbns_00` to see the exact spelling of its categories (for example, `wbns_00 |> tabyl(FPL_LT50)`). Your filter must match the category spelling exactly, including capital letters.
2. Write your filter using the conditions in your project's tab below. Remember from Lesson 23: `&` means **both** conditions must be true, and `|` means **at least one** must be true. Use parentheses to group the "or" part.
3. Save the result as `wbns_01_pop`.
4. Use `dim()` on `wbns_00` and on `wbns_01_pop` to compare the number of rows before and after filtering.

Project 1

Keep respondents who are under 100% of the FPL **and** received free groceries or meals in the past 12 months:

- Under 100% FPL: `FPL_LT50` **or** `FPL50_100` equals `"Yes"`
- **and** received free groceries or meals: **any** of `Q25`, `Q25A`, `Q26`, or `Q26A` equals `"Yes"`

Look these four questions up in the questionnaire: `Q25` and `Q26` ask about free groceries and free meals in the past 30 days, and `Q25A` and `Q26A` follow up with people who said no. These are exactly the people who were asked the `Q26D` ease-of-access questions.

The structure of your filter will be: `(FPL condition | FPL condition) & (Q25 condition | Q25A condition | Q26 condition | Q26A condition)`.

Project 2

Keep respondents who identify as LGBTQ **and** who have a healthcare provider:

- `LGBTQ` equals `"Yes"`
- **and** `Q108` was **not** answered `"No"`

For the second condition, use `filter_out()` from Lesson 25 instead of `!=`. Remember, `filter(Q108 != "No")` would also silently drop anyone with a missing `Q108`. You'll look at that missing data on purpose in the "Explore missing data" section.

If you chose a different population (for example, using `Q70F`), write out your conditions in words first, then translate them into code.

Project 3

Keep respondents with at least one child aged 5-18 in the household **and** who are under 100% of the FPL:

- `ANY_CHILDHH_5_18` equals `"Yes"`
- **and** `FPL_LT50` **or** `FPL50_100` equals `"Yes"`

The structure of your filter will be: `ANY_CHILDHH_5_18 condition & (FPL condition | FPL condition)`.

> **!** Task
>
> Write a pipeline that filters the dataset down to your specific population of interest. Save this as a new object called `wbns_01_pop`. Use `dim()` to print the number of rows before and after filtering to verify the operation.

> Caution
>
> Please make sure you are referencing the PDF with the questionnaire when filtering out your population. There are important descriptions and caveats in the questionnaire!

Selecting Project Variables

Your dataset contains hundreds of variables. For project clarity, reduce your data frame to only the variables in the informational table above.

1. Make a list of the variables you need from the table: the respondent ID variable (find its name in the codebook), your **population** variables, your **main** variables, and the **additional** variables.
2. *Optional:* If you are interested in other variables, add 0-3 more that are not listed.
3. Use `select()` to keep only those variables. You can type each name, or use a helper from Lesson 23 like `starts_with("Q26D")` to grab a group of similarly named variables at once.
4. Save the result as `wbns_02_pop_vars`.
5. Use `dim()` on `wbns_01_pop` and on `wbns_02_pop_vars`. The number of rows should stay the same, and the number of columns should drop.

> **!** Task
>
> Use `select()` to keep the variables relevant to your project. Save this as a new object called `wbns_02_pop_vars`. Use `dim()` to print the number of columns before and after selecting to verify the operation.

Rename variables for clarity

Most variables in the WBNS dataset are named after their survey question code (e.g. `Q108B_2`), which isn't very informative on its own and is easy to mix up as your pipeline grows. Renaming your variables to short, descriptive names now will make the rest of your code (and your missing data plots) easier to read and less error-prone.

1. Use `rename()` with the pattern `new_name = old_name`.
2. Rename your **main** variables using the **exact names** in your project's tab below. Labs 3 and 4 use these names.
3. Rename your **additional** variables. Some have names given below. For any listed as "your choice," pick a name that follows these rules:
   - all lowercase, with words separated by underscores (_), and no spaces
   - short, but describes what the question asks (look it up in the questionnaire!)
4. You do **not** need to rename the ID or population variables.
5. Save the result as `wbns_03_renamed`, and use `glimpse()` to confirm your new names.

Project 1

Original	New name	What it measures
`FOOD_INSEC`	`food_insecure`	Household food insecurity
`Q26D_1`	`ease_find_place`	Ease of finding a place to get free groceries/meals
`Q26D_2`	`ease_transport`	Ease of finding transportation
`Q26D_3`	`ease_variety`	Ease of getting a variety of food
`Q26D_4`	`ease_timely`	Ease of getting there in a reasonable time
`PPETHM`	`race_ethnicity`	Race/ethnicity
`NUM_CHILD_HH`	`num_children`	Number of children in the household
`Q23`, `Q37`	your choice	Look these up in the questionnaire

Project 2

Original	New name	What it measures
`Q108B_1`	`courtesy_respect`	"I am treated with courtesy and respect by my personal health care provider"
`Q108B_2`	`trust_provider`	"I trust my personal health care provider"
`Q27`	`self_rated_health`	Self-reported health
`Q70F_4`	`unfair_gender_identity`	Treated unfairly at a health care visit because of gender identity

Original	New name	What it measures
Q70F_5	unfair_sexual_orientation	Treated unfairly at a health care visit because of sexual orientation
PPETHM	race_ethnicity	Race/ethnicity
Q28O	your choice	Look this up in the questionnaire

Project 3

Mental health items:

Original	New name	In the past 30 days, how often did you feel…
Q30_1	mh_nervous	nervous
Q30_2	mh_hopeless	hopeless
Q30_3	mh_restless	restless or fidgety
Q30_4	mh_sad	so sad that nothing could cheer you up
Q30_5	mh_effort	that everything was an effort
Q30_6	mh_worthless	worthless

Benefit items with skip logic (these come in pairs; you'll combine each pair when you build your composite score in Lab 3):

Original	New name	What it measures
Q53_B	medicaid_chip_q53	Received Medicaid/CHIP (asked in Q53)
Q31_4	medicaid_chip_q31	Covered by Medicaid/CHIP (asked earlier, in Q31 item d)
Q53_C	housing_q53	Received housing assistance (asked in Q53)
Q11	housing_q11	Paying lower rent because of a government program (asked earlier, in Q11)

Other benefit items:

Original	New name	Program
Q53_A	benefit_snap	SNAP (food stamps)
Q53_D	benefit_ssi	Supplemental Security Income (SSI)
Q53_E	benefit_ssdi	Social Security Disability Income (SSDI)

Original	New name	Program
Q53_F	benefit_tanf	Cash assistance from a state or county welfare program, including TANF
Q53_G	benefit_unemployment	Unemployment insurance
Q53_H	benefit_school_meals	Free or reduced-price school meals
Q53_I	benefit_wic	WIC

Additional variables:

Original	New name
NUM_CHILD_HH	num_children
PPETHM	race_ethnicity
Q27	self_rated_health
Q1, Q37	your choice

> **!** Task
>
> Use `rename()` to give your **main** and **additional** variables clear, descriptive names, using the required names for your project in the tables above. Save this as a new object called `wbns_03_renamed`. Use `glimpse()` to confirm your new variable names.

Explore missing data

Before we change or remove anything, we need to know **how much data is missing, where it is, and why.** We'll use the `naniar` functions from Lesson 24. All of these steps use `wbns_03_renamed`, and none of them create a new dataset.

There are **two kinds** of missing data to look for:

- **True `NA`s:** R already knows these are missing. The `naniar` functions find them.
- **Hidden missing values:** responses like `"Refused"` or `"Don't know"`. R treats these as real categories, so the `naniar` functions will **not** count them. We find these with `tabyl()`.

1. **Overall missingness.** Use `n_miss()` and `prop_miss()` on `wbns_03_renamed` to get the total number and proportion of missing cells in your dataset.
2. **Missingness by variable.** Use `miss_var_summary()`, and pipe the result into `gt()` so it prints as a table. Which variables have the most missing data?

3. **Visualize missingness.** Use `vis_miss()` to see where the `NA`s are across rows and columns. If your variable names overlap, add the `theme()` line from Lesson 24 to angle them.
4. **Missing together?** Use `gg_miss_upset()` (with `nsets = 10`) to see which variables tend to be missing *together*.
5. **Hidden missing values.** Use `tabyl()` on **each main variable**, one at a time. Write down any categories that really mean "no answer" (for example, `"Refused"` or `"Don't know"`). You'll need this list in the next section.
6. **Explain what you found.** Follow your project's tab below, then write your paragraph.

Project 1

The `ease_` items come from `Q26D`, which is only asked of people who received free groceries or meals. Since your population filter already requires that, these items should be mostly complete. If you see a lot of missingness, double-check your filter.

You'll also see a lot of missing data in `Q25A` and `Q26A`. That's skip logic, and it doesn't mean "no." `Q25A` (free groceries in the past **12 months**) is only asked of people who said no to `Q25` (free groceries in the past **30 days**). If someone got free groceries in the past 30 days, they obviously got them in the past 12 months, so the survey skips `Q25A`. `Q26A` works the same way with `Q26` for free meals. Your filter already handles this: because it uses `|`, a `"Yes"` on `Q25` keeps the person even when `Q25A` is `NA`.

The `ease_` items also have a `"Don't know"` response option. Look for it in your `tabyl()` output!

Project 2

You kept respondents with a missing `Q108` on purpose (that's why we used `filter_out()`). In your paragraph, say how many there are and how missing they are on `courtesy_respect`, `trust_provider`, and `self_rated_health`.

Use `gg_miss_fct()` to compare missingness across levels of `Q108` (set `fct = Q108`). Does missingness on your main variables look different for people who have a provider and people whose `Q108` is missing?

Project 3

Some of your benefit variables will look **very** missing. Much of that missing data is **skip logic**, not refusals: people were never asked the question because of an earlier answer. Let's see it for ourselves.

Use `gg_miss_fct()` with `fct = medicaid_chip_q31`. Look at the `medicaid_chip_q53` row: people who said `"Covered"` on Q31 should be (almost) 100% missing on Q53, because they were skipped past it.

Then do the same with `fct = housing_q11` and look at the `housing_q53` row: people who said `"Yes"` on Q11 should be (almost) 100% missing on Q53_C. Notice that `housing_q11` itself is missing for many people too. Q11 is only asked of people who rent or live without paying rent (Q8), so homeowners skip it. Those people **were** asked Q53_C.

Also note how much `benefit_wic` is missing. WIC is only asked of households with a child aged 0-5, and your population doesn't require that.

> **!** Task
>
> Explore the missing data in `wbns_03_renamed`:
>
> 1. Use `n_miss()` and `prop_miss()` to report the overall missingness of your dataset.
> 2. Use `miss_var_summary()` (piped into `gt()`) to show missingness for each variable.
> 3. Create a `vis_miss()` plot and a `gg_miss_upset()` plot.
> 4. Complete the project-specific step from your project's tab (Projects 2 and 3: create the `gg_miss_fct()` plot(s)).
> 5. Use `tabyl()` on each main variable to find "hidden" missing values like `"Refused"` or `"Don't know"`.
>
> Write a brief paragraph (4-6 sentences) summarizing your findings. Which variables have the most missing data? Do some variables tend to be missing together? Do you think the missingness is mostly from refusals, from skip logic (people who were never asked), or something else? Which hidden missing values will you need to turn into `NA`?

Handle missing data

Now that we know what's missing and why, we'll handle it in two parts:

- **Part A:** Turn hidden missing values (like `"Refused"`) into real `NA`s so R treats them as missing.
- **Part B:** Remove rows that are missing on your key variables.

We do this now, **before** you build your composite score in Lab 3, because `NA` will disrupt arithmetic. For example, `5 + NA` returns `NA`, not `5`. And if we left `"Refused"` as a category, it would be counted as a real answer.

Part A: Turn hidden missing values into `NA`

1. Use `replace_with_na()` from Lesson 25 on `wbns_03_renamed`. Inside `replace = list()`, list each variable and the categories you found in the "Explore missing data" section. For example:

```
wbns_04_na_recoded <- wbns_03_renamed |>
  replace_with_na(replace = list(
    variable_1 = c("Refused", "Don't know"),
    variable_2 = "Refused"
  ))
```

   Replace `variable_1`, `variable_2`, and the categories with your own. Only list categories that really appear in your `tabyl()` output, and spell them exactly as they appear. **Watch the apostrophe in "Don't know"!** The data may use a curly apostrophe (') instead of a straight one ('). If they don't match, `replace_with_na()` silently changes nothing. The safest move is to copy the category name straight from your `tabyl()` output.

2. Save the result as `wbns_04_na_recoded`.

3. **Check your work:** use `tabyl()` on at least one variable you changed, and re-run `miss_var_summary()`. The number missing should go up by the number of refusals you recoded. If it didn't change, the spelling in `replace_with_na()` doesn't match the data. In `tabyl()`, you may still see a `"Don't know"` row with a count of 0. That's fine: the category name is still listed, but no one is in it anymore.

Part B: Remove rows missing your key variables

4. Use `drop_na()` on `wbns_04_na_recoded`, listing **only** the key variables named in your project's tab below inside the parentheses. Do **not** use `drop_na()` with nothing inside the parentheses! That would remove anyone missing *any* variable, including additional variables you don't need for your composite score.
5. Save the result as `wbns_04_no_na`.
6. Use `dim()` on `wbns_04_na_recoded` and `wbns_04_no_na` to compare the number of rows.

Project 1

Part A: Recode `"Don't know"` (and any refusals you found) to `NA` in the four `ease_` items and `food_insecure`.

Part B: Drop rows missing any of these: `food_insecure`, `ease_find_place`, `ease_transport`, `ease_variety`, `ease_timely`.

Do not drop rows based on `Q25A` or `Q26A`. They are missing because of skip logic, and dropping them would remove people who got free food in the past 30 days.

Project 2

Part A: Recode any refusals you found to `NA` in `courtesy_respect`, `trust_provider`, and `self_rated_health`.

Part B: Drop rows missing any of these: `courtesy_respect`, `trust_provider`, `self_rated_health`.

Project 3

Part A: Recode `"Don't know"` to `NA` in all seven `benefit_` variables, `medicaid_chip_q53`, `housing_q53`, and `housing_q11`. Recode `"Not Sure"` to `NA` in `medicaid_chip_q31` (Q31 uses "Not Sure" instead of "Don't know"). The `mh_` items have no "Don't know" option, but recode any other non-answers you found with `tabyl()`.

Skipped questions are already `NA` in the data (there is no "Skipped" category), so you don't need to recode anything for skip logic.

Part B: Drop rows missing any of the six `mh_` items, and any of the benefit variables that **everyone** in your population was asked: `benefit_snap`, `benefit_ssi`, `benefit_ssdi`, `benefit_tanf`, `benefit_unemployment`, and `benefit_school_meals`.

Do not drop rows based on `medicaid_chip_q53`, `medicaid_chip_q31`, `housing_q53`, `housing_q11`, or `benefit_wic`. Those are missing mostly because of skip logic. You'll handle them when you build the composite score in Lab 3.

> **!** Task
>
> **Part A:** Use `replace_with_na()` to turn any "Refused" or "Don't know" responses into `NA`. Save this as `wbns_04_na_recoded`. Check your work with `tabyl()` and `miss_var_summary()`.
> **Part B:** Use `drop_na()` to remove rows with missing values in the key variables listed for your project. Save this as `wbns_04_no_na`. Use `dim()` to print the number of rows before and after this step. Write 1-2 sentences describing how many rows you lost and whether that seems reasonable.

Export datasets

It's important to save your work! We want to export the datasets we created along the way so we can use them in future labs and in our final project. We'll save them all into **one** `.rda` file, which can hold several datasets at once.

1. Give `export()` a `list()` of your datasets. Inside `list()`, each dataset gets a name with the pattern `name = object`. Use the same name as the object.
2. Use `here()` to save the file in your `data_project/data_processed` folder.

Your code will look like this:

```
export(
  list(
    wbns_01_pop = wbns_01_pop,
    wbns_02_pop_vars = wbns_02_pop_vars,
    wbns_03_renamed = wbns_03_renamed,
    wbns_04_no_na = wbns_04_no_na
  ),
  here("data_project", "data_processed", "wbns_data_lab_02_work.rda")
)
```

3. Check your `data_processed` folder to make sure the file is there.

> **!** Task
>
> Save `wbns_01_pop`, `wbns_02_pop_vars`, `wbns_03_renamed`, and `wbns_04_no_na` in a `.rda` file in your `data_project/data_processed` folder. Name the file something that will help you identify it later, like `wbns_data_lab_02_work.rda` or `wbns_data_02_clean.rda`.

AI Use Statement

Using AI is not required in this class. If you *did* use an AI tool (ChatGPT, Claude, Copilot, Gemini, an RStudio assistant, etc.) on this lab, add a brief AI Use Statement. Include:

- **Which tool(s)** you used
- **What for** (a sentence or two, with section names)
- **Where** it contributed to any written answers (if at all)
- *You do NOT need to link your conversation.*

For example:

> *AI Use Statement: Claude was used to explain an error message in the "Handle missing data" section. All reasoning and answers are my own.*

OR

> *AI Use Statement: AI was not used in this assignment.*

Before using AI, check the table of allowed uses in the Generative AI Policy. AI can help you explain errors, debug, and style your code, but it should not write your analysis for you.

! Task

Write your AI Use Statement in your editing file.